TUTORIAL DETALHADO

Conversão de Tokens em AST usando Gramáticas Livres de Contexto

Disciplina: Compiladores
Contexto: analisador léxico, analisador sintático e linguagem MINIC

Material didático acadêmico

Sumário

1. OBJETIVO DO TUTORIAL 5
2. CONCEITOS FUNDAMENTAIS 5
2.1 Lexema, token e atributo 5
2.2 Sintaxe concreta e sintaxé abstrata 6
2.3 O que é uma Gramática Livre de Contexto 6
3. A RELAÇÃO ENTRE TOKENS, TERMINAIS E PRODUÇÕES 6
4. ESCOLHENDO UMA GRAMÁTICA ADEQUADA PARA A AST 7
4.1 Gramática ambígua ou inadequada 7
4.2 Gramática com precedência é associatividade 7
5. MODELO DE AST 8
5.1 Exemplo de estrutura conceitual 8
6. O PARSER COMO CONSUMIDOR DE TOKENS 9
7. CONSTRUÇÃO DA AST COM DESCIDA RECURSIVA 10
7.1 Regra Factor 10
7.2 Regra Term 11
7.3 Regra Expr 11
8. EXEMPLO COMPLETO: EXPRESSÃO ARITMÉTICA 11
8.1 Tokens gerados pelo scanner 12
1. IDENT("a") 12
2. PLUS("+") 12
3. INT_LITERAL(3) 12
4. STAR("*") 12
5. LPAREN("(") 12
6. IDENT("b") 12
7. MINUS("-") 12
8. INT_LITERAL(1) 12
9. RPAREN(")") 12
10. EOF 12
8.2 Percurso do parser 12
8.3 AST resultante 12
9. COMO A GRAMÁTICA PRODUZ A ESTRUTURA DA AST 13
10. ATRIBUIÇÕES E COMANDOS 13
10.1 Diferenciando atribuição de igualdade 14
11. DECLARAÇÕES DE VARIÁVEIS 14

12. BLOCOS, IF, WHILE E RETURN 15
12.1 Bloco . . . 15
12.2 Comando condicional . . . 15
12.3 Comando de repetição . . . 15
12.4 Retorno . . . 15
13. CHAMADAS DE FUNÇÃO E LISTAS DE ARGUMENTOS 16
14. LISTAS, REPETIÇÃO E A REGRA EPSILON 16
15. FIRST, FOLLOW E DECISÃO DA PRODUÇÃO 17
15.1 FIRST . . . 17
15.2 FOLLOW . . . 17
15.3 Escolha de epsilon . . . 18
16. PARSER PREDITIVO COM TABELA LL(1) 18
16.1 Pilha e valores semânticos . . . 18
17. DERIVACAO, RECONHECIMENTO E CONSTRUÇÃO: NÃO CONFUNDIR 18
1. Derivacao: aplicação teórica de produções da gramática. 19
2. Reconhecimento: verificação de que os tokens pertencem à línguagem. 19
3. Construção: criação dos objetos AST duranté o reconhecimento. 19
18. EXEMPLO COMPLETO DE UM PROGRAMA MINIC 19
18.1 Tokens simplificados . . . 19
18.2 Estrutura sintática esperada . . . 19
18.3 AST resultante . . . 20
18.4 Ordem de construção . . . 20
1. Reconhece KW_INT e registra o tipo INT. 20
2. Reconhece soma e cria o nome da funcao. 20
3. Reconhece os parametros a e b. 20
4. Entra no bloco ao consumir LBRACE. 20
5. Reconhece a declaracao de resultado. 20
6. Reconhece a atribuicao e constrói a soma. 20
7. Reconhece o retorno. 20
8. Consome RBRACE. 20
9. Cria o no Funcao completo. 20
10. Adiciona a função ao nó Programa. 20
19. TRATAMENTO DE ERROS SINTÁTICOS 20
19.1 Recuperacao pelo modo pânico . . . 21
20. ERROS COMUNS NA CONVERSÃO PARA AST 21
20.1 Ignorar a precedência . . . 21

20.2 Construir a árvore na ordem errada . . . 21
20.3 Consumir um token duas vezes . . . 21
20.4 Não preservar a posição do erro . . . 21
20.5 Confundir menós unário com menós binário . . . 22
20.6 Colocar detalhes léxicos na AST . . . 22
21. ALTERNATIVA: AST CONCRETA E AST ABSTRATA 22
22. TESTANDO O PARSER E A AST 22
22.1 Testes de expressões . . . 22
22.2 Testes de comandos . . . 23
22.3 Testes negativos . . . 23
22.4 Forma de comparar ASTs . . . 23
23. ROTEIRO DE IMPLEMENTAÇÃO 23
1. Liste todos os tipos de token produzidos pelo scanner. 24
2. Defina a gramatica concreta da linguagem. 24
3. Elimine recursao a esquerda. 24
4. Faça fatoracao a esquerda quando necessario. 24
5. Organize as regras para explicitar precedencia. 24
6. Calcule ou verifique FIRST e FOLLOW. 24
7. Defina os tipos de nos da AST. 24
8. Implemente as funcoes de leitura e consumo de tokens. 24
9. Implemente Factor, Term e Expr. 24
10. Implementé atribuições e declarações. 24
11. Implemente blocos e comandos de controle. 24
12. Implemente funções, parâmetros e chamadas. 24
13. Inclua linha e coluna nós nós e nós erros. 24
14. Adicione recuperação de erros. 24
15. Imprima a AST em formato legivel. 24
16. Crie testes positivos e negativos. 24
17. Comparé as ASTs produzidas com ASTs esperadas. 24
24. CHECKLIST FINAL 24
25. CONCLUSÃO 25

1. OBJETIVO DO TUTORIAL

Este tutorial explica, passo a passo, como transformar a sequência de tokens produzida por um analisador léxico em uma Arvore Sintatica Abstrata (AST, do inglês Abstract Syntax Tree), útilizando uma Gramática Livre de Contexto (GLC).

A conversão é realizada pelo analisador sintático, também chamado de parser. O parser recebe tokens, verifica se eles obedecem a sintaxe da línguagem e, ao mesmo tempo, constrói uma estrutura em árvore que representa o significado sintático do programa.

O fluxo geral de um compilador pode ser resumido assim:

```
codigo-fonte
    |
    v
analisador lexico (scanner)
    |
    v
tokens
    |
    v
analisador sintatico (parser baseado em uma GLC)
    |
    v
AST
    |
    v
analise semantica, interpretacao ou geracao de codigo
```

O foco aqui esta na etapa intermediaria:

```
TOKENS  --->  PARSER + GLC  --->  AST
```

A AST não é uma copia literal do texto-fonte. Ela remove detalhes que não são necessários para as etapas posteriores, como parênteses usados apenas para agrupamento, comentarios e, em geral, separadores. Ao mesmo tempo, preserva a estrutura essencial das expressões, comandos, declarações e chamadas de função.

2. CONCEITOS FUNDAMENTAIS

2.1 Lexema, token e atributo

O scanner percorre os caracteres do código-fonte e agrupa sequências de caracteres em unidades chamadas lexemas. Para cada lexema, produz um token.

Um token normalmente possui:

```
tipo       categoria sintatica do lexema;
lexema     texto original encontrado;
atributo   valor associado ao token, quando necessario;
linha      linha em que o token aparece;
coluna     coluna inicial do token.
```

Exemplo de código:

```
total = 10 + valor;
```

Uma representação possível dos tokens e:

```
IDENT(total)  ATRIB  INT_LITERAL(10)  PLUS  IDENT(valor)  EOF
```

Uma representação mais detalhada poderia ser:

```
{tipo: IDENT, lexema: "total", valor: "total", linha: 1, coluna: 1}
{tipo: ATRIB, lexema: "=", linha: 1, coluna: 7}
```

```
{tipo: INT_LITERAL, lexema: "10", valor: 10, linha: 1, coluna: 9}
{tipo: PLUS, lexema: "+", linha: 1, coluna: 12}
{tipo: IDENT, lexema: "valor", valor: "valor", linha: 1, coluna: 14}
{tipo: EOF, lexema: "", linha: 1, coluna: 19}
```

O parser não deve voltar a analisar caracteres. Ele trabalha sobre os tipos de token e, quando necessário, usa os atributos para preencher nós da AST.

2.2 Sintaxe concreta e sintaxé abstrata

A sintaxe concreta descreve como o programa e escrito. Por exemplo, uma expressão pode ser escrita como:

```
(a + b) * c
```

A sintaxé abstrata representa a estrutura relevante:

```
        MUL
       /   \
     ADD    c
    /   \
   a     b
```

Na AST, os parênteses não precisam ser armazenados como nós. Eles já cumpriram sua função: orientar a construção da estrutura. O resultado da árvore mostra que a soma deve ser realizada antes da multiplicação.

2.3 O que é uma Gramática Livre de Contexto

Uma GLC é definida formalmente por uma quadrupla:

```
G = (N, T, P, S)
```

onde:

```
N = conjunto de simbolos nao-terminais;
T = conjunto de simbolos terminais;
P = conjunto de producoes;
S = simbolo inicial.
```

Os terminais correspondem, nó parser, aos tipos de token. Os não-terminais representam catégorias sintáticas, como expressão, comando ou declaração.

Exemplo:

```
N = {Expr, Term, Factor}
T = {IDENT, INT_LITERAL, PLUS, STAR, LPAREN, RPAREN}
S = Expr
```

Producoes:

```
Expr   -> Term ExprTail
ExprTail -> PLUS Term ExprTail | epsilon
Term   -> Factor TermTail
TermTail -> STAR Factor TermTail | epsilon
Factor -> IDENT | INT_LITERAL | LPAREN Expr RPAREN
```

O símbolo epsilon significa uma derivação vazia: o nao-terminal pode ser removido sem consumir token.

3. A RELAÇÃO ENTRE TOKENS, TERMINAIS E PRODUÇÕES

O scanner produz tokens. A GLC usa terminais. A ligacao entré os dois e feita por uma tabela conceitual:

```
token do scanner       terminal da gramatica
-----------------------------------------------
IDENT                  IDENT
INT_LITERAL             INT_LITERAL
PLUS                   PLUS
STAR                   STAR
LPAREN                 LPAREN
RPAREN                 RPAREN
EOF                    EOF
```

Não é necessário que o nome internó sejá idêntico, mas a correspondência deve ser clara e única.

Exemplo de entrada:

```
x + 3 * y
```

Sequencia de tokens:

```
IDENT(x), PLUS, INT_LITERAL(3), STAR, IDENT(y), EOF
```

A gramática reconhece essa sequência usando as produções de Expr, Term e Factor. A cada passo, o parser decide qual produção aplicar e qual token consumir.

A regra mais importante e:

```
terminais consomem tokens;
nao-terminais organizam o reconhecimento;
producoes determinam a estrutura sintatica;
acoes semanticas constroem os nos da AST.
```

4. ESCOLHENDO UMA GRAMÁTICA ADEQUADA PARA A AST

Uma gramática pode reconhecer umà línguagem e, ainda assim, ser inadequada para um parser preditivo. Antes de implementar o parser, e necessário verificar:

```
- se ha recursao a esquerda;
- se ha alternativas com prefixo comum;
- se a precedencia dos operadores esta representada;
- se a associatividade desejada esta representada;
- se os tokens da gramatica correspondem aos tokens do scanner.
```

4.1 Gramática ambígua ou inadequada

Uma tentativa comum para expressões e:

```
Expr -> Expr PLUS Expr
      | Expr STAR Expr
      | LPAREN Expr RPAREN
      | IDENT
      | INT_LITERAL
```

Essa gramática possui recursão a esquerda e é ambígua. Para a sequência:

```
a + b * c
```

ela pode gerar estruturas diferentes, sem informar claramente sé a soma ou a multiplicação deve ser realizada primeiro.

4.2 Gramática com precedência é associatividade

Uma forma adequada para os operadores +, -, * e / e:

```
Expr       -> Term ExprTail
ExprTail   -> PLUS Term ExprTail
            | MINUS Term ExprTail
            | epsilon
```

```
Term       -> Factor TermTail
TermTail   -> STAR Factor TermTail
            | SLASH Factor TermTail
            | epsilon

Factor     -> LPAREN Expr RPAREN
            | IDENT
            | INT_LITERAL
            | REAL_LITERAL
            | MINUS Factor
```

Essa organizacao estabelece:

```
- Expr trata adicao e subtracao;
- Term trata multiplicacao e divisao;
- Factor trata operandos, parenteses e menos unario;
- multiplicacao e divisao possuem precedencia maior que adicao e
  subtracao;
- parenteses permitem alterar a ordem;
- a regra de repeticao define a associatividade.
```

5. MODELO DE AST

Antes de escrever o parser, defina os tipos de nós que poderao aparecer. Para umà línguagem MINIC, um modelo inicial pode conter:

```
Programa
DeclaracaoVariavel
DeclaracaoFuncao
Parametro
Bloco
If
While
Return
ExpressaoStmt
Atribuicao
Binaria
Unaria
Chamada
Identificador
LiteralInteiro
LiteralReal
LiteralCaractere
LiteralString
```

Cada nó devé armazenar apenas as informações necessárias.

5.1 Exemplo de estrutura conceitual

```
NoAST
  - tipo
  - linha
  - coluna

NoBinario
  - operador
  - esquerda
  - direita

NoUnario
  - operador
  - operando

NoLiteral
  - valor

NoIdentificador
  - nome

NoAtribuicao
  - nome ou alvo
  - expressao
```

Em uma implementação em C, uma estrutura possível seria:

```
typedef enum {
    AST_PROGRAMA,
    AST_BINARIO,
    AST_UNARIO,
    AST_LITERAL_INT,
    AST_LITERAL_REAL,
    AST_IDENT,
    AST_ATRIBUICAO,
    AST_BLOCO,
    AST_IF,
    AST_WHILE,
    AST_RETURN,
    AST_CHAMADA
} TipoAST;

typedef struct AST AST;

struct AST {
    TipoAST tipo;
    int linha;
    int coluna;
    union {
        int inteiro;
        double real;
        char *nome;
        struct {
            TokenTipo operador;
            AST *esquerda;
            AST *direita;
        } binario;
        struct {
            TokenTipo operador;
            AST *operando;
        } unario;
    } dados;
};
```

O formato exato depende do projeto. O ponto essencial é que os nós possuem filhos que representam subexpressões ou subcomandos.

6. O PARSER COMO CONSUMIDOR DE TOKENS

O parser mantém uma posição sobré o vetor ou fluxo de tokens. Normalmente são necessárias tres operações básicas:

```
olhar()       consulta o token atual sem consumi-lo;
avancar()     consome o token atual e avanca;
aceitar(T)    consome T se o token atual for T;
esperar(T)    exige T; produz erro se o token for diferente.
```

Pseudocódigo:

```
Token atual;

Token olhar() {
    return atual;
}

void avancar() {
    atual = proximo_token();
}

bool aceitar(TokenTipo tipo) {
    if (atual.tipo == tipo) {
        avancar();
        return true;
    }
    return false;
}
```

```
Token esperar(TokenTipo tipo) {
    if (atual.tipo != tipo) {
        erro_sintatico("token inesperado");
    }
    Token consumido = atual;
    avancar();
    return consumido;
}
```

A função esperar e importante porque centraliza a verificação sintática e permite apresentar mensagens com linha, coluna, lexema encontrado e tipo esperado.

7. CONSTRUÇÃO DA AST COM DESCIDA RECURSIVA

Na descida recursiva, cada nao-terminal da gramática costuma corresponder a uma função do parser.

Para a gramática:

```
Expr     -> Term ExprTail
Term     -> Factor TermTail
Factor   -> IDENT | INT_LITERAL | LPAREN Expr RPAREN
```

podemos criar:

```
AST *parse_expr();
AST *parse_term();
AST *parse_factor();
```

7.1 Regra Factor

Pseudocódigo:

```
AST *parse_factor() {
    if (aceitar(MINUS)) {
        AST *filho = parse_factor();
        return criar_unario(MINUS, filho);
    }

    if (token_atual() == INT_LITERAL) {
        Token t = esperar(INT_LITERAL);
        return criar_literal_inteiro(t.valor, t.linha, t.coluna);
    }

    if (token_atual() == REAL_LITERAL) {
        Token t = esperar(REAL_LITERAL);
        return criar_literal_real(t.valor, t.linha, t.coluna);
    }

    if (token_atual() == IDENT) {
        Token t = esperar(IDENT);
        return criar_identificador(t.lexema, t.linha, t.coluna);
    }

    if (aceitar(LPAREN)) {
        AST *no = parse_expr();
        esperar(RPAREN);
        return no;
    }

    erro_sintatico("era esperado identificador, literal ou '('");
    return NULL;
}
```

Observe que os parênteses são consumidos, mas não geram necessáriamente um no. A subárvore criada por parse_expr já representa o agrupamento.

7.2 Regra Term

A gramática de Term possui uma sequência de fatores ligados por * ou /. Uma implementação simples e:

```
AST *parse_term() {
    AST *esquerda = parse_factor();

    while (token_atual() == STAR || token_atual() == SLASH) {
        Token operador = atual;
        avancar();
        AST *direita = parse_factor();
        esquerda = criar_binario(operador.tipo,
                                 esquerda,
                                 direita,
                                 operador.linha,
                                 operador.coluna);
    }

    return esquerda;
}
```

A atualização de esquerda é o que produz a associatividadé a esquerda. Para:

```
a / b / c
```

as etapas são:

```
esquerda = a
esquerda = (a / b)
esquerda = ((a / b) / c)
```

7.3 Regra Expr

A regra de Expr e semelhante, mas trabalha com + e -:

```
AST *parse_expr() {
    AST *esquerda = parse_term();

    while (token_atual() == PLUS || token_atual() == MINUS) {
        Token operador = atual;
        avancar();
        AST *direita = parse_term();
        esquerda = criar_binario(operador.tipo,
                                 esquerda,
                                 direita,
                                 operador.linha,
                                 operador.coluna);
    }

    return esquerda;
}
```

Como parse_expr chama parse_term, e parse_term chama parse_factor, a precedência e implementada pela própria estrutura das funções.

8. EXEMPLO COMPLETO: EXPRESSÃO ARITMÉTICA

Entrada-fonte:

```
a + 3 * (b - 1)
```

8.1 Tokens gerados pelo scanner

1. IDENT("a")

2. PLUS("+")

3. INT_LITERAL(3)

4. STAR("*")

5. LPAREN("(")

6. IDENT("b")

7. MINUS("-")

8. INT_LITERAL(1)

9. RPAREN(")")

10. EOF

8.2 Percurso do parser

```
parse_expr()
  -> parse_term()
    -> parse_factor()
      consome IDENT("a")
    retorna Ident(a)
  encontra PLUS
  -> parse_term()
    -> parse_factor()
      consome INT_LITERAL(3)
    retorna Literal(3)
    encontra STAR
    -> parse_factor()
      consome LPAREN
      -> parse_expr()
        -> parse_term()
          -> parse_factor()
            consome IDENT("b")
          retorna Ident(b)
        encontra MINUS
        -> parse_term()
          -> parse_factor()
            consome INT_LITERAL(1)
          retorna Literal(1)
        cria Binario(MINUS, b, 1)
      consome RPAREN
    retorna Binario(MINUS, b, 1)
    cria Binario(STAR, 3, (b - 1))
  cria Binario(PLUS, a, (3 * (b - 1)))
```

8.3 AST resultante

```
Binario(PLUS)
|-- Identificador(a)
`-- Binario(STAR)
    |-- LiteralInteiro(3)
    `-- Binario(MINUS)
        |-- Identificador(b)
        `-- LiteralInteiro(1)
```

Representacao parentetizada:

```
(PLUS (IDENT a)
      (STAR (INT 3)
            (MINUS (IDENT b) (INT 1))))
```

Essa AST preserva a precedência correta: primeiro b - 1, depois 3 * o resultado, e por fim a + o resultado da multiplicação.

9. COMO A GRAMÁTICA PRODUZ A ESTRUTURA DA AST

Uma derivação possível para a expressão a + 3 * (b - 1) e:

```
Expr
=> Term ExprTail
=> Factor TermTail ExprTail
=> IDENT TermTail ExprTail
=> IDENT ExprTail
=> IDENT PLUS Term ExprTail
=> IDENT PLUS Factor TermTail ExprTail
=> IDENT PLUS INT_LITERAL STAR Factor TermTail ExprTail
=> IDENT PLUS INT_LITERAL STAR LPAREN Expr RPAREN
   TermTail ExprTail
=> IDENT PLUS INT_LITERAL STAR LPAREN Term ExprTail RPAREN
   TermTail ExprTail
=> IDENT PLUS INT_LITERAL STAR LPAREN Factor TermTail ExprTail RPAREN
   TermTail ExprTail
=> ...
```

A derivação mostra o reconhecimento. A implementação do parser, porem, não precisa construir uma árvore de derivação completa. Em vez disso, cria diretamenté a AST ao reduzir cada parte relevante:

```
Factor -> IDENT       cria Identificador
Factor -> INT_LITERAL cria LiteralInteiro
Term   -> Factor ...  cria nos STAR ou SLASH
Expr   -> Term ...    cria nos PLUS ou MINUS
```

A AST é, portanto, uma árvore menor e mais significativa que a árvore de derivação completa.

10. ATRIBUIÇÕES E COMANDOS

Umà línguagem MINIC normalmente possui comandos dé atribuição. Uma gramática simples pode ser:

```
Stmt -> IDENT ASSIGN Expr SEMI
```

Exemplo:

```
total = preco + imposto;
```

Tokens:

```
IDENT(total) ASSIGN IDENT(preco) PLUS IDENT(imposto) SEMI EOF
```

Parser:

```
AST *parse_stmt() {
    Token nome = esperar(IDENT);
    esperar(ASSIGN);
    AST *valor = parse_expr();
    esperar(SEMI);
    return criar_atribuicao(nome.lexema, valor,
                            nome.linha, nome.coluna);
}
```

AST:

```
Atribuicao
|-- alvo: Identificador(total)
`-- valor: Binario(PLUS)
            |-- Identificador(preco)
            `-- Identificador(imposto)
```

O token ASSIGN não precisa ser um nó isolado. Ele informa ao parser que a produção dé atribuição esta sendo reconhecida. O operador e representado pelo nó Atribuicao.

10.1 Diferenciando atribuição de igualdade

Sé à línguagem possui ==, a gramática deve diferenciar:

```
Stmt -> IDENT ASSIGN Expr SEMI
Expr -> Equality
Equality -> Relational EqualityTail
EqualityTail -> EQ Relational EqualityTail | epsilon
```

Assim:

```
x = y == 10;
```

gera uma atribuição cujo valor é uma comparacao:

```
Atribuicao(x,
  Binario(EQ, Ident(y), Literal(10)))
```

A ordem das camadas da gramática deve refletir a precedência dos operadores.

11. DECLARAÇÕES DE VARIÁVEIS

Exemplo de produções:

```
Decl -> Type IDENT DeclInit SEMI
DeclInit -> ASSIGN Expr | epsilon
Type -> INT | FLOAT | CHAR
```

Entrada:

```
int contador = 0;
```

Tokens:

```
KW_INT IDENT(contador) ASSIGN INT_LITERAL(0) SEMI EOF
```

Parser:

```
AST *parse_decl() {
    TipoDados tipo = parse_type();
    Token nome = esperar(IDENT);
    AST *inicializador = NULL;

    if (aceitar(ASSIGN)) {
        inicializador = parse_expr();
    }

    esperar(SEMI);
    return criar_declaracao(tipo, nome.lexema,
                            inicializador,
                            nome.linha, nome.coluna);
}
```

AST:

```
DeclaracaoVariavel(tipo=INT, nome=contador)
`-- inicializador: LiteralInteiro(0)
```

Se não houver inicializador, o campo pode ser NULL:

```
int contador;

DeclaracaoVariavel(tipo=INT, nome=contador)
```

```
`-- inicializador: NULL
```

12. BLOCOS, IF, WHILE E RETURN

12.1 Bloco

```
Block -> LBRACE StmtList RBRACE
StmtList -> Stmt StmtList | epsilon
```

O parser cria um nó Bloco com uma lista ordenada de filhos.

Entrada:

```
{
    x = 1;
    y = x + 2;
}
```

AST:

```
Bloco
|-- Atribuicao(x, Literal(1))
`-- Atribuicao(y, Binario(PLUS, x, Literal(2)))
```

As chaves são consumidas, mas geralmente não se tornam nós da AST.

12.2 Comando condicional

```
Stmt -> IF LPAREN Expr RPAREN Stmt ElsePart
ElsePart -> ELSE Stmt | epsilon
```

Entrada:

```
if (x > 0) y = 1; else y = 2;
```

AST:

```
If
|-- condicao: Binario(GT, Ident(x), Literal(0))
|-- entao: Atribuicao(y, Literal(1))
`-- senao: Atribuicao(y, Literal(2))
```

12.3 Comando de repetição

```
Stmt -> WHILE LPAREN Expr RPAREN Stmt
```

Entrada:

```
while (i < 10) i = i + 1;
```

AST:

```
While
|-- condicao: Binario(LT, Ident(i), Literal(10))
`-- corpo: Atribuicao(i,
             Binario(PLUS, Ident(i), Literal(1)))
```

12.4 Retorno

```
Stmt -> RETURN ReturnExpr SEMI
ReturnExpr -> Expr | epsilon
```

Entradas:

```
return valor;
return;
```

ASTs:

```
Return
`-- Identificador(valor)

Return
```

```
`-- NULL
```

13. CHAMADAS DE FUNÇÃO E LISTAS DE ARGUMENTOS

Uma produção para chamadas pode ser:

```
Factor -> IDENT CallPart
CallPart -> LPAREN ArgList RPAREN | epsilon
ArgList -> Expr ArgTail | epsilon
ArgTail -> COMMA Expr ArgTail | epsilon
```

Entrada:

```
calcula(a + 1, 20)
```

Tokens principais:

```
IDENT(calcula) LPAREN IDENT(a) PLUS INT_LITERAL(1)
COMMA INT_LITERAL(20) RPAREN
```

AST:

```
Chamada
|-- funcao: Identificador(calcula)
|-- argumento 1: Binario(PLUS, Ident(a), Literal(1))
`-- argumento 2: Literal(20)
```

Pseudocódigo:

```
AST *parse_call_or_ident() {
    Token nome = esperar(IDENT);
    AST *base = criar_identificador(nome.lexema,
                                    nome.linha,
                                    nome.coluna);

    if (!aceitar(LPAREN)) {
        return base;
    }

    Lista argumentos = lista_vazia();
    if (token_atual() != RPAREN) {
        adicionar(&argumentos, parse_expr());
        while (aceitar(COMMA)) {
            adicionar(&argumentos, parse_expr());
        }
    }

    esperar(RPAREN);
    return criar_chamada(base, argumentos,
                         nome.linha, nome.coluna);
}
```

A alternativa vazia em ArgList permite chamadas sem argumentos:

```
inicia();
```

AST:

```
Chamada
|-- funcao: Identificador(inicia)
`-- argumentos: lista vazia
```

14. LISTAS, REPETIÇÃO E A REGRA EPSILON

Producoes com epsilon são muito comuns:

```
Lista -> Item Lista | epsilon
```

O parser deve interpretar epsilon como "não ha mais itens". Ele não consome token para reconhecer epsilon.

Exemplo:

```
StmtList -> Stmt StmtList | epsilon
```

A função correspondente pode ser:

```
Lista parse_stmt_list() {
    Lista lista = lista_vazia();

    while (inicio_de_stmt(token_atual())) {
        adicionar(&lista, parse_stmt());
    }

    return lista;
}
```

A condição início_de_stmt pode usar o conjunto FIRST de Stmt. Por exemplo, se um comando pode iniciar com IDENT, IF, WHILE, RETURN ou LBRACE:

```
FIRST(Stmt) = {IDENT, IF, WHILE, RETURN, LBRACE}
```

Quando o token atual não pertencé a FIRST(Stmt), o parser encerra a lista e deixa o chamador consumir o token de fechamento, como RBRACE.

15. FIRST, FOLLOW E DECISÃO DA PRODUÇÃO

Para um parser preditivo, os conjuntos FIRST e FOLLOW ajudam a decidir qual produção deve ser usada.

15.1 FIRST

FIRST(X) é o conjunto de terminais que podem iniciar alguma derivação de X.

Para:

```
Factor -> IDENT | INT_LITERAL | LPAREN Expr RPAREN
```

temos:

```
FIRST(Factor) = {IDENT, INT_LITERAL, LPAREN}
```

Se houver menós unário:

```
Factor -> MINUS Factor | IDENT | INT_LITERAL | LPAREN Expr RPAREN
```

entao:

```
FIRST(Factor) = {MINUS, IDENT, INT_LITERAL, LPAREN}
```

15.2 FOLLOW

FOLLOW(A) é o conjunto de terminais que podem aparecer imediatamente após o nao-terminal A em alguma derivação.

Em geral:

```
EOF pertence a FOLLOW(Expr)
```

se Expr é o símbolo inicial. Em uma produção:

```
Factor -> LPAREN Expr RPAREN
```

RPAREN pertencé a FOLLOW(Expr), pois Expr aparecé antes de RPAREN.

Para:

```
Expr -> Term ExprTail
```

ExprTail pode terminar quando aparecem tokens de FOLLOW(Expr), como RPAREN, SEMI, COMMA ou EOF, dependendo do contexto dà línguagem.

15.3 Escolha de epsilon

Considere:

```
ExprTail -> PLUS Term ExprTail | MINUS Term ExprTail | epsilon
```

Sé o token atual for PLUS, escolhe-sé a primeira alternativa. Se for MINUS, escolhe-sé a segunda. Sé o token atual pertencer a FOLLOW(ExprTail), escolhe-se epsilon.

Essa regra evita que o parser tente consumir um operador quando a expressão já terminou.

16. PARSER PREDITIVO COM TABELA LL(1)

Em um parser LL(1), a decisão usa:

```
L: leitura da entrada da esquerda para a direita;
L: construcao de uma derivacao mais a esquerda;
1: apenas um token de antecipacao.
```

A tabela M[A, a] informa qual produção usar quando o topo da pilha é o nao-terminal A é o token atual é a.

Exemplo parcial para ExprTail:

```
M[ExprTail, PLUS]  = ExprTail -> PLUS Term ExprTail
M[ExprTail, MINUS] = ExprTail -> MINUS Term ExprTail
M[ExprTail, RPAREN] = ExprTail -> epsilon
M[ExprTail, SEMI]   = ExprTail -> epsilon
M[ExprTail, EOF]    = ExprTail -> epsilon
```

O parser de tabela pode usar uma pilha de símbolos é uma pilha paralela dé atributos semânticos. Contudo, em muitos projetos didáticos, a descida recursiva e mais simples para construir a AST.

16.1 Pilha e valores semânticos

Quando uma produção é aplicada, o parser empilha os símbolos do lado direito. Quando um terminal e consumido, seu valor pode ser associado a um registro semântico. Quando todos os componentes dé uma produção são processados, uma acao semântica combina os valores e cria o nó da AST.

Exemplo abstrato:

```
E -> T E'
```

Depois de reconhecer T e E', a acao pode ser:

```
valor(E) = combinar(valor(T), valor(E'))
```

Na prática, para expressões com listas dé operadores, e comum usar uma lista intermediaria de pares operador/operando e depois dobrar a lista para a esquerda.

17. DERIVACAO, RECONHECIMENTO E CONSTRUÇÃO: NÃO CONFUNDIR

Tres ideias devem ser separadas:

1. Derivacao: aplicação teórica de produções da gramática.

2. Reconhecimento: verificação de que os tokens pertencem à línguagem.

3. Construção: criação dos objetos AST duranté o reconhecimento.

Um parser correto realiza as tres tarefas de forma integrada, mas elas não são a mesma coisa.

Exemplo:

```
Expr -> Expr PLUS Term
```

A produção ajuda a definir uma soma, mas a AST não precisa guardar um nó chamado Expr nem os símbolos intermediarios. Depois de reconhecer os filhos, o parser cria:

```
Binario(PLUS, esquerda, direita)
```

Ações semânticas devem ser associadas aos pontos em que os componentes necessários já estao disponíveis.

18. EXEMPLO COMPLETO DE UM PROGRAMA MINIC

Considere:

```
int soma(int a, int b) {
    int resultado;
    resultado = a + b;
    return resultado;
}
```

18.1 Tokens simplificados

```
KW_INT
IDENT(soma)
LPAREN
KW_INT IDENT(a)
COMMA
KW_INT IDENT(b)
RPAREN
LBRACE
KW_INT IDENT(resultado) SEMI
IDENT(resultado) ASSIGN IDENT(a) PLUS IDENT(b) SEMI
RETURN IDENT(resultado) SEMI
RBRACE
EOF
```

18.2 Estrutura sintática esperada

```
Program -> DeclList EOF
DeclList -> Decl DeclList | epsilon
Decl -> Type IDENT FunctionPart
FunctionPart -> LPAREN Params RPAREN Block
Params -> Param ParamTail | epsilon
ParamTail -> COMMA Param ParamTail | epsilon
Param -> Type IDENT
Block -> LBRACE StmtList RBRACE
```

18.3 AST resultante

```
Programa
`-- Funcao
    |-- nome: soma
    |-- parametros
    |   |-- Parametro(tipo=INT, nome=a)
    |   `-- Parametro(tipo=INT, nome=b)
    `-- corpo: Bloco
        |-- DeclaracaoVariavel(tipo=INT, nome=resultado)
        |-- Atribuicao(resultado,
        |     Binario(PLUS, Ident(a), Ident(b)))
        `-- Return(Ident(resultado))
```

18.4 Ordem de construção

1. Reconhece KW_INT e registra o tipo INT.

2. Reconhece soma e cria o nome da funcao.

3. Reconhece os parametros a e b.

4. Entra no bloco ao consumir LBRACE.

5. Reconhece a declaracao de resultado.

6. Reconhece a atribuicao e constrói a soma.

7. Reconhece o retorno.

8. Consome RBRACE.

9. Cria o no Funcao completo.

10. Adiciona a função ao nó Programa.

19. TRATAMENTO DE ERROS SINTÁTICOS

O parser deve rejeitar tokens que não podem ser derivados pela GLC. Uma mensagem útil deve informar:

```
- linha e coluna;
- token encontrado;
- lexema encontrado;
- tokens esperados;
- contexto sintatico, quando possivel.
```

Entrada com erro:

```
total = 10 + ;
```

Tokens:

```
IDENT(total) ASSIGN INT_LITERAL(10) PLUS SEMI EOF
```

Ao chamar parse_factor depois de PLUS, o token atual e SEMI. O parser pode emitir:

```
Erro sintatico na linha 1, coluna 14: era esperado identificador,
literal, '(' ou expressão unaria; encontrado SEMI (';').
```

19.1 Recuperacao pelo modo pânico

Uma estratégia simples e descartar tokens até encontrar um ponto de sincronização, por exemplo:

```
SEMI, RBRACE, EOF
```

Pseudocódigo:

```
void sincronizar() {
    while (token_atual() != SEMI &&
           token_atual() != RBRACE &&
           token_atual() != EOF) {
        avancar();
    }
    if (token_atual() == SEMI) {
        avancar();
    }
}
```

A recuperação permite diagnósticar mais de um erro em uma compilacao, em vez de interromper nó primeiro problema. Entretanto, uma AST parcial não deve ser tratada como valida para as etapas seguintes sem uma marca explicita de erro.

20. ERROS COMUNS NA CONVERSÃO PARA AST

20.1 Ignorar a precedência

Se todos os operadores forem tratados em uma única função, a entrada:

```
a + b * c
```

pode ser transformada incorretamente em:

```
(a + b) * c
```

Solucao: separar Expr, Term e Factor, ou usar uma tecnica equivalente como precedence climbing ou Pratt parser.

20.2 Construir a árvore na ordem errada

Para associatividadé a esquerda, a cada operador deve-sé atualizar o nó acumulado:

```
esquerda = Binario(op, esquerda, direita)
```

Sé o parser sempre guardar a última operação sem incorporar o resultado anterior, pode produzir uma árvore direita quando à línguagem espera uma árvore esquerda.

20.3 Consumir um token duas vezes

Uma função deve ter responsabilidade clara sobre os tokens que consome. Se parse_factor consome RPAREN e parse_term também tenta consumi-lo, o parser perdé a sincronização.

Regra prática: a função correspondenté a uma produção deve consumir os terminais dessa produção, é o chamador não deve repeti-los.

20.4 Não preservar a posição do erro

Cada nó AST e cada erro deve, quando possível, preservar linha e coluna. Isso ajuda na análise semântica e nas mensagens posteriores.

20.5 Confundir menós unário com menós binário

Compare:

```
-x
a - x
```

O primeiro deve produzir:

```
Unario(MINUS, Ident(x))
```

O segundo:

```
Binario(MINUS, Ident(a), Ident(x))
```

A gramática é o parser devem tratar essas ocorrencias em contextos diferentes.

20.6 Colocar detalhes léxicos na AST

Comentarios, espacos e parênteses de agrupamento normalmente não devem virar nós da AST. Ja a informação de valor de um literal deve ser preservada, pois sera usada depois.

21. ALTERNATIVA: AST CONCRETA E AST ABSTRATA

Alguns compiladores constroem primeiro uma árvore sintática concreta (CST), que preserva todos os símbolos da derivação, e depois fazem uma transformacao para AST.

CST para:

```
a + 3
```

pode conter nós Expr, Term, Factor, ExprTail é os terminais PLUS.

AST correspondente:

```
Binario(PLUS, Ident(a), Literal(3))
```

Vantagens da CST:

```
- facilita depuracao do parser;
- preserva detalhes da sintaxe concreta;
- pode ser util para formatadores e ferramentas de origem.
```

Vantagens de construir AST diretamente:

```
- usa menos memoria;
- simplifica as etapas posteriores;
- evita uma segunda travessia para remover nos artificiais.
```

Para um projeto didático de MINIC, construir a AST diretamente durante o parsing costuma ser uma escolha adequada.

22. TESTANDO O PARSER E A AST

Os testes devem verificar tanto a aceitação ou rejeicao da entrada quanto a estrutura da AST produzida.

22.1 Testes de expressões

Entradas recomendadas:

```
42
x
-x
a + b
```

```
a + b * c
(a + b) * c
a / b / c
f(x, 2)
```

Para cada entrada, confira:

```
- tokens consumidos na ordem correta;
- EOF alcançado ao final;
- operadores nos niveis corretos;
- associatividade correta;
- valores literais preservados.
```

22.2 Testes de comandos

```
x = 10;
if (x > 0) x = 1;
while (x < 10) x = x + 1;
return x;
{ x = 1; y = 2; }
```

22.3 Testes negativos

```
x = ;
x = 1 + ;
if x > 0) y = 1;
while (x < 10) {
int = 3;
f(1, );
```

O resultado esperado deve indicar erro sintático, sem produzir uma AST silenciosamente incorreta.

22.4 Forma de comparar ASTs

Uma maneira simples e legivel e serializar a AST em uma forma parentetizada:

```
(ASSIGN x (PLUS (INT 1) (STAR (INT 2) (INT 3))))
```

Entrada:

```
x = 1 + 2 * 3;
```

Saida esperada:

```
(ASSIGN x (PLUS (INT 1) (STAR (INT 2) (INT 3))))
```

Essa representação e conveniente para testes automatizados porque permite comparar texto, embora a implementação use ponteiros e estruturas em memória.

23. ROTEIRO DE IMPLEMENTAÇÃO

Uma sequência recomendada para implementar a conversão e:

1. Liste todos os tipos de token produzidos pelo scanner.

2. Defina a gramatica concreta da linguagem.

3. Elimine recursao a esquerda.

4. Faça fatoracao a esquerda quando necessario.

5. Organize as regras para explicitar precedencia.

6. Calcule ou verifique FIRST e FOLLOW.

7. Defina os tipos de nos da AST.

8. Implemente as funcoes de leitura e consumo de tokens.

9. Implemente Factor, Term e Expr.

10. Implementé atribuições e declarações.

11. Implemente blocos e comandos de controle.

12. Implemente funções, parâmetros e chamadas.

13. Inclua linha e coluna nós nós e nós erros.

14. Adicione recuperação de erros.

15. Imprima a AST em formato legivel.

16. Crie testes positivos e negativos.

17. Comparé as ASTs produzidas com ASTs esperadas.

24. CHECKLIST FINAL

Antes de considerar o parser concluído, confirme:

```
[ ] Cada token do scanner possui correspondencia na gramatica.
[ ] O parser consome EOF ao aceitar um programa completo.
[ ] Nao existe recursao a esquerda nas regras usadas pelo parser LL.
[ ] A precedencia dos operadores esta refletida na AST.
[ ] A associatividade dos operadores esta correta.
[ ] Parenteses alteram a estrutura quando necessario.
[ ] Literais preservam seus valores.
[ ] Identificadores preservam nome e posicao.
[ ] Atribuicoes nao sao confundidas com comparacoes.
[ ] Menos unario e menos binario sao diferenciados.
[ ] Blocos preservam a ordem dos comandos.
```

```
[ ] Chamadas preservam a ordem dos argumentos.
[ ] Erros informam linha, coluna e token encontrado.
[ ] A recuperacao nao entra em loop infinito.
[ ] Entradas invalidas nao sao aceitas silenciosamente.
[ ] A AST pode ser impressa ou serializada para testes.
```

25. CONCLUSÃO

A conversão de tokens em AST é uma aplicação direta das Gramáticas Livres de Contexto nó processo de compilacao. O scanner identifica as unidades léxicais; a GLC define como essas unidades podem ser combinadas; o parser reconhecé a sequência e executa acoes semânticas que constroem a AST.

A qualidade da AST depende de tres decisões principais:

```
- uma gramatica sem ambiguidades relevantes;
- um parser que consuma os tokens com disciplina;
- um modelo de nos que represente a estrutura necessaria as etapas
  seguintes.
```

Para expressões, a separacao Expr, Term e Factor é uma tecnica didática e eficaz para representar precedência é associatividade. Para comandos, a mesma ideia se estendé a declarações, blocos, condicionais, laços, retornós e chamadas de função.

Ao final, a AST funciona como uma representação intermediaria estruturada do programa. Ela pode ser percorrida para verificar tipos, resolver identificadores, interpretar a MINIC ou gerar código de máquina virtual, assembly ou outrà línguagem-alvo.

FIM DO TUTORIAL